



PUSL3190 Computing Project

Project Interim Report

OncoStage AI

Supervisor: Ms.Nimesha Hewawasam

Name: A.R.Nanayakkara

Plymouth Index Number: 10953370

Degree Program: Msc(hons) Computer Science

Table of Contents

1	Chapter 01 - Introduction.....	4
1.1	Introduction	4
1.1.1	Background of the Domain	4
1.1.2	Why this Problem is Important	4
1.1.3	Industry / Academic Relevance.....	4
1.2	Problem Definition.....	5
1.2.1	Heavy Reliance on Imaging and Offline Resources	5
1.2.2	Inefficient Patient-Specialist Allocation	5
1.2.3	Communication Gaps and Delayed Staging	5
1.3	Project Objective	6
2	System Analysis.....	7
2.1	Fact Gathering Techniques	7
2.1.1	Proof of Fact Gathering	7
2.2	Existing System	8
2.3	Drawbacks of the existing system.....	9
3	Requirement Specification.....	10
3.1	Functional Requirements	10
3.2	Non Functional Requirements	10
3.3	Hardware/Software Requirements.....	11
4	Feasibility Study	12
4.1	Operational Feasibility	12
4.2	Economical Feasibility	12
4.3	Technical Feasibility	13
5	System Architecture	14
5.1	Use case diagram	14
5.2	Class Diagram	15
5.3	ER Diagram	16
5.4	High Level Architecture Diagram	17
6	Development Tools and Technologies	18
6.1	Development Technology.....	18
6.2	Programming Language and Tools	18
6.3	Third party components and libraries	19
6.4	Algorithms.....	19
7	Implementation Progress	20
7.1	Development Environment Set up	20
7.2	Implemented Features	21
7.2.1	Authentication & Security	21

7.2.2	Healthcare Management	21
7.2.3	Backend & API Integration	21
7.3	Screenshots	22
7.4	Challenges Encountered and Solutions	26
7.4.1	Handling Imbalanced Medical Data:	26
7.4.2	Managing Doctor Patient Balancing:	26
7.4.3	Asynchronous AI Inference:	26
7.5	Current System Limitations	27
8	Discussion	28
8.1	Summary of the Report	28
8.2	What has Changed from the proposal	28
8.3	Future Plans	28
9	References	29

List Of Figures

Figure 5.1:Use case Diagram.....	14
Figure 5.2:Class Diagram.....	15
Figure 5.3:ER Diagram.....	16
Figure 5.4:High Level Architecture Diagram	17

1 Chapter 01 - Introduction

1.1 Introduction

Lung cancer has been one of the most prevalent and lethal diseases worldwide, primarily due to the late diagnosis of the disease and the lack of proactive diagnostic tools. With the current advancements in the healthcare system, there is a large amount of clinical data available, such as patient symptoms, smoking habits, and demographic data. However, clinical data patterns are not being effectively utilized in real time predictive systems compared to traditional systematic processes. OncoStage AI is an innovative clinical decision support system that offers an accessible web-based tool for lung cancer risk and stage prediction.

1.1.1 Background of the Domain

In the current medical environment, diagnosis of the disease primarily depends on medical image processing or laboratory tests. However, there has been a significant increase in the use of Artificial Intelligence and Machine Learning to utilize clinical data patterns to identify disease risk.

1.1.2 Why this Problem is Important

Delayed diagnosis of the disease has been found to be highly correlated with low efficiency in cure rates and high treatment costs. Both the patient and medical professionals face a major constraint due to the lack of an easily accessible prediction tool in the traditional health process.

1.1.3 Industry / Academic Relevance

Academically, it has been identified that there is a research gap for multi class classification of cancer stages using non image clinical data. In industry, there is an urgent shift towards adopting Model-View-Controller (MVC) architecture and containerization (Docker) to ensure that medical AI applications are scalable and portable

1.2 Problem Definition

The major problem that this project attempts to solve is that of an efficient, easily accessible, and automated system for predicting lung cancer stages using clinical data, which has a direct impact on the speed of medical decision-making processes. This problem is defined under three major pillars as follows:

1.2.1 Heavy Reliance on Imaging and Offline Resources

Most existing lung cancer prediction models heavily rely on complex medical image analysis (CT scans or MRI scans), offline laboratory tests, etc. In many medical environments, especially those that are resource-constrained, the lack of easy access to these diagnostic tools for predicting lung cancer stages poses a major hindrance to a patient's journey to recovery. There is a clear absence of web-based models that can utilize structured data such as patient symptoms, medical history, or even patient smoking habits for early-stage predictions.

1.2.2 Inefficient Patient-Specialist Allocation

Managing student records manually, akin to an industry parallel, and interpreting clinical data manually are error-prone and not scalable. Moreover, in oncology, there exists no centralized system that manages the allocation of work among specialists in real time. As a result, without an automated system that allocates patient records to doctors based on certain capacity constraints, such as 10 patients per week, specialists get overwhelmed, thus leading to a further delay in the availability of vital medical information to the patient.

1.2.3 Communication Gaps and Delayed Staging

The incidence rate of lung cancer is generally accompanied by delays in diagnosis at a late stage. In the current system, there exists no unified system that enables the instant sharing of AI-generated reports with the patient and the doctor. This leads to:

Data Inconsistency: Reports are not centralized, making historical trends difficult to track.

Delayed Consultation: A significant time gap exists between the generation of a prediction result and the subsequent consultation call, thus leading to an increase in patient anxiety and a delay in the administration of life-saving medications.

1.3 Project Objective

The main objective of the OncoStage AI project is to create a fully functional prototype of an application that demonstrates the possibility of using an AI-based clinical decision support system, considering the university's limitations. With the application of an agile software development process, where small development cycles are used, the project will be accomplished by achieving the following objectives:

- To develop an application that uses a machine learning algorithm trained on Kaggle datasets to predict the stages of lung cancer or the risk of developing lung cancer with an accuracy of 85% or more.
- To develop a full-stack web application that has different interfaces for patients, medical professionals (doctors), and administrators.
- To develop an application that automates the assignment of specialists, ensuring that when a user logs in for the first time, they are required to choose a doctor who is within their maximum capacity of 10 patients per week.
- To develop an application that has a patient data input feature, where users are allowed to input their clinical attributes, which are necessary for making predictions.
- To develop a secure doctor dashboard through which doctors can view and validate AI-generated prediction result and patient information.
- To develop a feature for real-time communication through which patients and doctors can communicate for further medications and clarifications.
- To improve the performance of the system through which the AI model can display prediction results and reports within a time span of 5 seconds after the submission of the data.
- To ensure ethical and secure handling of the data through the incorporation of anonymized datasets and authentication and authorization features.

2 System Analysis

2.1 Fact Gathering Techniques

To ensure the OncoStage AI system is based on real-world needs in the clinical domain and academia, the fact-gathering approach of Document Analysis was used. Document analysis is a fact-gathering technique in which a review of available literature and research is conducted.

2.1.1 Proof of Fact Gathering

The following are the pieces of evidence collected from the research phase that validate the creation of the system:

- **Identification of Clinical Data:** Based on the analysis of available literature and datasets from Kaggle, specific data was identified as a high-impact attribute for the diagnosis of lung cancer. These data are age, gender, smoking status, and symptoms.
- **Identification of Research Gaps:** Based on the analysis of available literature from Google Scholar, IEEE Xplore, and PubMed, the absence of a multi-class classification model for the staging of cancer was identified.
- **Architectural Standards:** Based on the analysis of available literature from modern software used in the health sector, the Model-View-Controller (MVC) architecture and Docker are the industry standards for the development of health software.
- **Benchmarking Success Metrics:** Analysis of similar academic studies helped establish an accuracy level baseline of 80-90% for preliminary risk assessment, which became the project's specific success metric: $\geq 85\%$ accuracy level.
- **Constraint Definition:** Review of clinical workload studies and resource management documents helped establish the business logic for limiting specialist resources to 10 patients/week to prevent burnout.

2.2 Existing System

The current diagnostic model mostly focuses on manual interpretations of clinical information and offline medical image analysis. The current model mostly faces issues of fragmentation because most of the current models are not web integrated and mostly function in a silo environment. This silo environment proves to be challenging for both patients and experts.

Current Limitations:

- **Manual Dependency:** Healthcare professionals face difficulties in identifying lung cancer in its early stages when they mostly rely on manual interpretations of clinical information.
- **Resource Fragmentation:** A huge amount of information related to demographics, smoking habits, and medical symptoms is being created. This information mostly remains untapped.
- **Access Bottlenecks:** There are issues of delayed clinical decisions because current models mostly require accessing lab facilities. This proves to be a challenge for those institutions that do not have advanced technology.
- **Coordination Gaps:** There are issues of delayed updates and a lack of automated workload management. This mostly affects the consultation process between patients and oncology specialists.

2.3 Drawbacks of the existing system

Based on the analysis of the present clinical practices and the study of the available documents, the following are the drawbacks of the present system, which makes the need for the OncoStage AI system a necessity:

- **Delayed Decision-Making:** The present heavy reliance on manual interpretation and offline lab results results in considerable delays in the determination of the stages of lung cancer.
- **Inaccessible Diagnostic Tools:** The present diagnostic tools are not web-integrated, which makes it difficult for the patients and the experts in the health field to access the prediction tools.
- **Inefficient Specialist Coordination:** The absence of an automated specialist allocation system results in the overburdening of the experts and the inefficient interaction between the patients and the experts.
- **Untapped Clinical Data:** There is a considerable amount of data collected in a structured format from the patients, which remains unexploited for the real-time prediction analysis.
- **Poor Scalability and Portability:** The present academic and clinical models are not well designed in terms of architecture, such as MVC, and are not containerized

3 Requirement Specification

3.1 Functional Requirements

- **Role-Based Access Control:** The system shall allow users to register and log in under three distinct roles: Patient, Doctor, and Admin, with access restricted to their respective dashboards.
- **Doctor Selection & Capacity Logic:** The system shall require first time patients to select a doctor and shall strictly enforce a limit of 10 patients per doctor per week.
- **Health Data Submission:** The system shall allow patients to submit a health form containing clinical attributes (e.g., smoking history, age, symptoms) required for prediction.
- **AI Staging Prediction:** The system shall process submitted health data through the trained ML model to predict the lung cancer stage.
- **Automated Result Distribution:** The system shall automatically send the prediction report to both the patient and their assigned doctor immediately after processing.
- **Communication Suite:** The system shall provide an integrated interface for patients and doctors to engage in text-based messaging and video calls.
- **Doctor Review Panel:** The system shall allow doctors to view the comprehensive reports and details of all patients assigned to them.
- **Administrative Management:** The system shall allow administrators to manage user accounts, monitor system health, and oversee all system-wide activities.

3.2 Non Functional Requirements

- **Accuracy:** The AI prediction model shall maintain an accuracy rate of at least 85% on validation datasets.
- **Performance:** The system shall generate and display the AI staging result within 5 seconds of the patient submitting their health form.
- **Security:** User data and medical reports shall be secured using password encryption (hashing) and role based access tokens (JWT) to prevent unauthorized access.
- **Usability:** The web application shall provide a responsive user interface (UI) that is easily navigable for both medical professionals and non technical patients across various devices.
- **Availability:** The system shall maintain consistent availability during peak testing hours with minimal runtime errors.

3.3 Hardware/Software Requirements

- **Server:** Dockerized environment hosted on a cloud-based VPS for consistent deployment and scalability.
- **Backend:** Node.js for managing the primary API and FastAPI for serving the AI model inference.
- **Frontend:** Angular framework for building a dynamic and responsive web application.
- **Database:** PostgreSQL for structured storage of user profiles, medical reports, and chat history.
- **Client:** Any modern web browser (Chrome, Firefox, Safari) on a desktop or mobile device.
- **Hardware (Development):** Minimum 8GB RAM and an i5 processor (or equivalent) for local model training and testing.

4 Feasibility Study

4.1 Operational Feasibility

This study aims to determine the effectiveness of OncoStage AI in terms of its integration into the daily workflow of patients and medical practitioners.

User Role Alignment: The system has a role based login system that allows only patients, doctors, or administrators to interact with the system, which makes it easier to manage the workflow.

Workload Management: On a weekly basis, the system allows each doctor to manage a maximum of 10 patients, which operationally manages the workload of doctors and prevents specialist fatigue.

Decision Support Integration: The system does not replace doctors in any way and acts as a supporting system to help doctors make a diagnosis, which makes it highly acceptable in a professional medical environment.

Accessibility: Being a web based system, it can be accessed at any location, which addresses the issue of limited availability of experts in health institutions.

4.2 Economical Feasibility

This will assess the cost effectiveness of the project, as well as the benefits of the project's implementation.

Open Source Cost Efficiency: The project has opted to use a cost-efficient set of technologies, such as Node.js, Angular, PostgreSQL, and FastAPI, which do not require the project to pay the high costs of purchasing proprietary software.

Research & Data Sourcing: By using publicly available datasets from Kaggle, the project has avoided the high costs of purchasing private hospital records.

Reduced Diagnostic Overhead: The project's ability to automate the staging process to less than 5 seconds has reduced the costs associated with the interpretation of the data.

Scalable Infrastructure: The project's use of Dockerized microservices allows the project to scale the resources used by the project without incurring the costs of cloud hosting

4.3 Technical Feasibility

This checks whether the current technology stack and skillset available are adequate to implement the proposed OncoStage AI feature set.

Proven Algorithmic Viability: An exhaustive study of the available literature on the Random Forest and SVM algorithms confirms that the proposed 85% accuracy target using structured clinical data is technically viable.

Full Stack: The proposed architecture has been successfully implemented to connect the React/Angular frontend to the Node.js backend using a separate FastAPI service to handle AI model inference.

Containerization: Using Docker to deploy the system ensures that the development environment mirrors the production environment, eliminating the infamous "it works on my machine" problem.

Data Security Protocols: Technical feasibility is also ensured through the implementation of JWT (JSON Web Tokens) and role based access control to handle medical data appropriately from a security perspective.

5 System Architecture

5.1 Use case diagram

The User Flow Diagram demonstrates the logical path and decision-making process for the key roles that operate within the system: Patient, Doctor, and Admin. The User Flow Diagram highlights the specific "First Time Login" logic, which identifies patients and allows them to select a doctor from the pool. The flow also demonstrates the progression from health form submission to AI-driven stage prediction and the opening of communication channels (chat/video) for consultation.

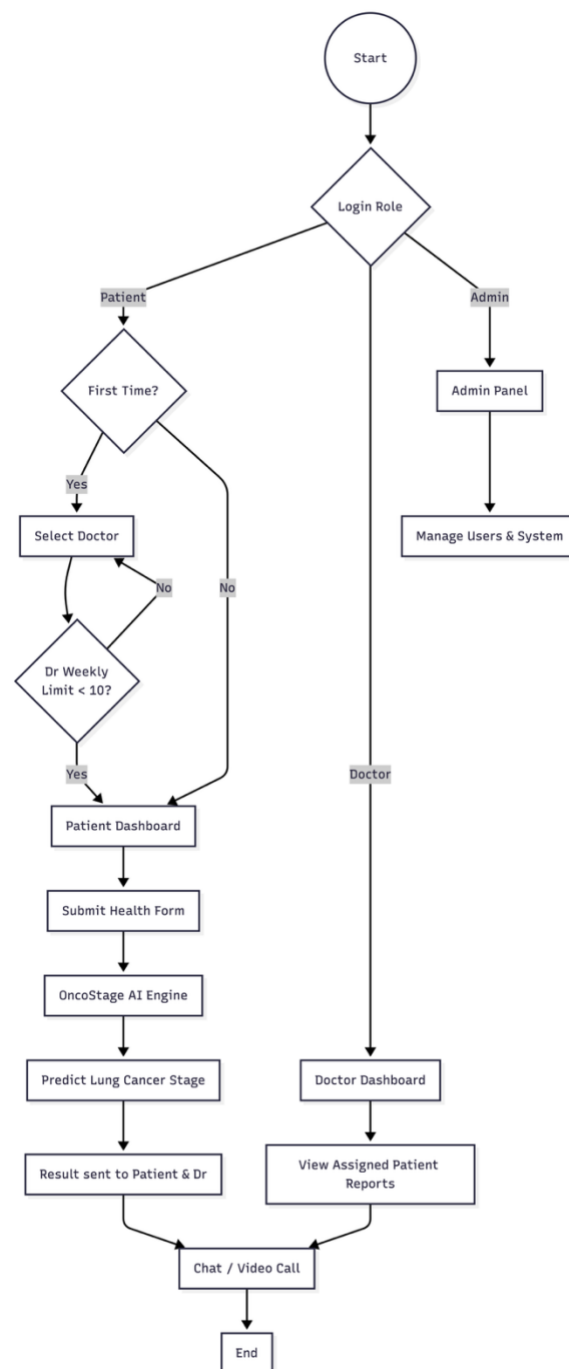


Figure 5.1: Use case Diagram

5.2 Class Diagram

The Class Diagram is an overview of the static structure of the objects in the system and their internal behaviors. It shows the attributes and methods of the Patient, Doctor, and Admin classes, along with the inheritance relationship with the User class. The predictStage(), assignDoctor(), and generateReport() methods are some of the primary methods of the OncoStage platform.

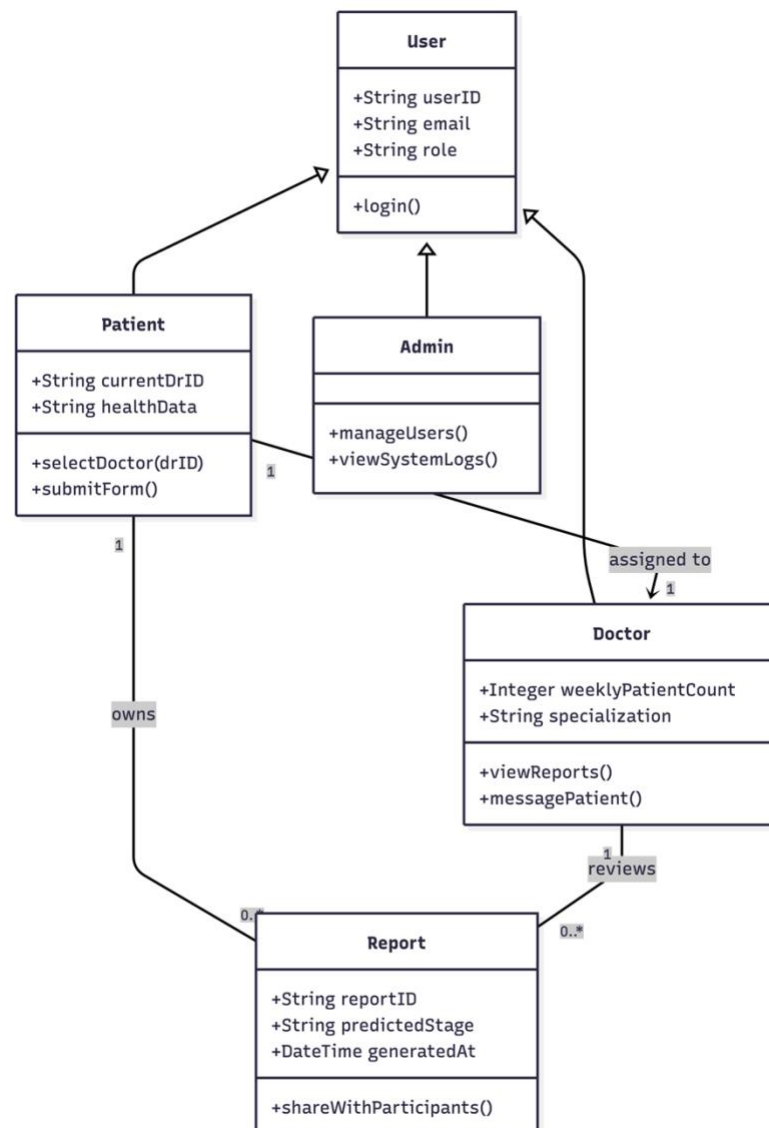


Figure 5.2:Class Diagram

5.3 ER Diagram

The ER diagram represents the structural data model and the relationships between different entities within the PostgreSQL database. A key part of this structural data model is the Doctor Patient relationship, which includes a counter to enforce the business rule of a maximum of 10 patients per week. It also shows the relationships between the User entities and the specialization using Role-Based Access Control (RBAC) for the Prediction Reports with the generating patient and the reviewing medical professional

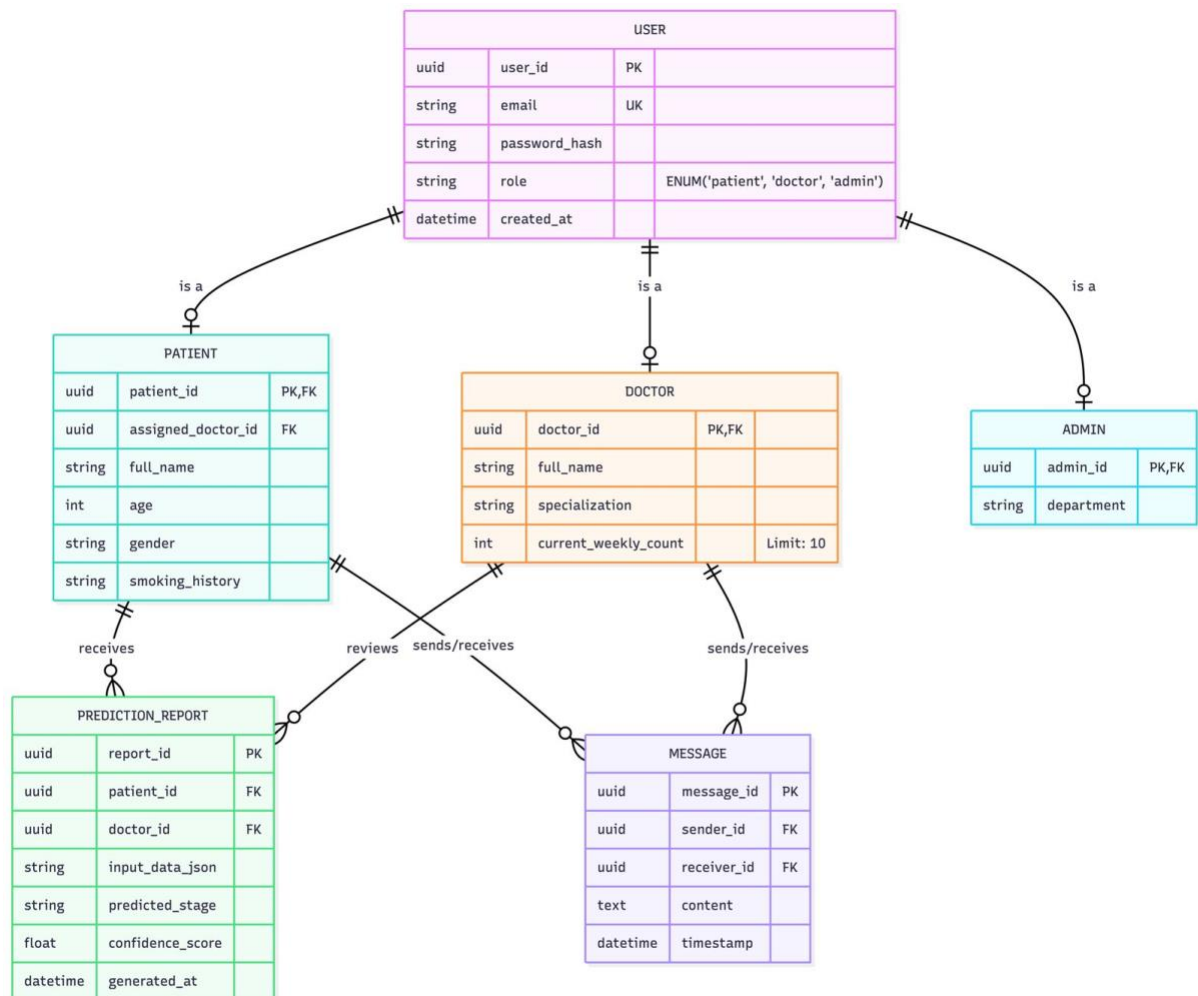


Figure 5.3:ER Diagram

5.4 High Level Architecture Diagram

The above diagram represents the Dockerized microservices architecture for the system, which highlights the separation of concern for the frontend, backend, and AI engine. The above architecture represents how the Angular frontend service communicates with the Node.js API gateway service, which further communicates with a FastAPI service for performing real-time Random Forest inferences. The architecture provides portability and scalability for the system, which meets the current industry standards for healthcare software deployment.

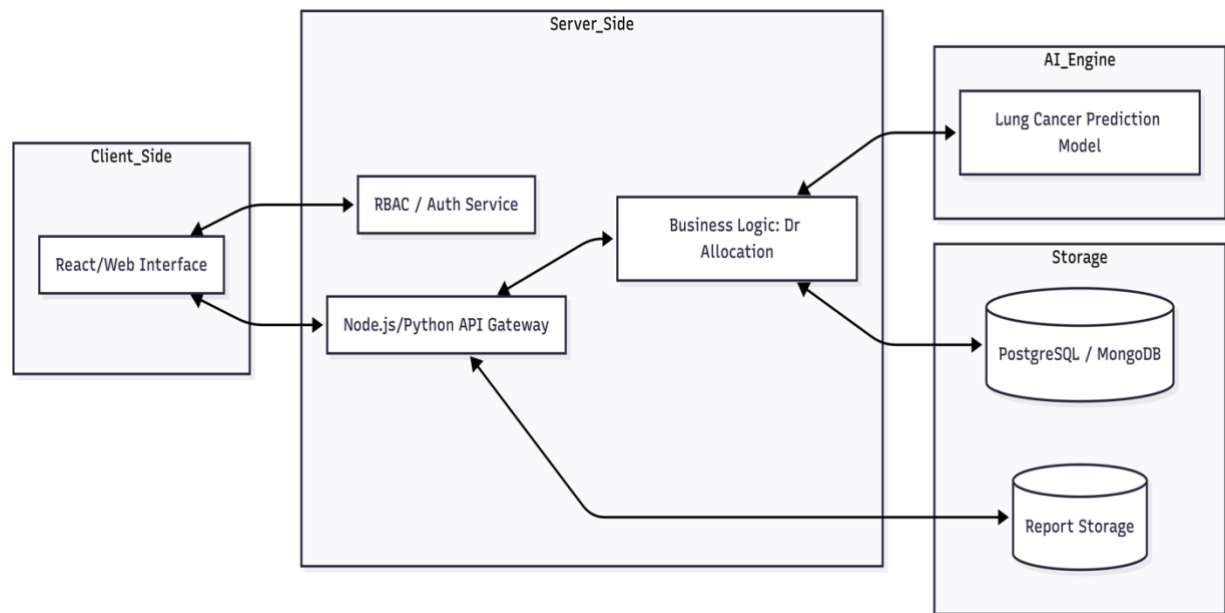


Figure 5.4: High Level Architecture Diagram

6 Development Tools and Technologies

6.1 Development Technology

The project is being executed following the Agile Methodology, adopting the iterative and incremental software development approach. Unlike the conventional "Waterfall" approach, in which refinement of the machine learning model is not possible, the Agile Methodology allows refinement of the model in parallel to the refinement of the web application infrastructure.

The importance of adopting this methodology in the development of the OncoStage AI project lies in the fact that the success of the project depends on the interplay between backend logic and the accuracy of the machine learning model. By adopting the incremental approach, in which the project is broken into short iterations or "sprints," it becomes possible to make the core business rules, including the role-based login and the specific constraint applied to doctors to restrict them to 10 patients a week, functional in a short time, thus enabling refinement of the project in response to the performance results of the Random Forest algorithm during the training phase.

6.2 Programming Language and Tools

The technology stack has been chosen in a way that it provides a clean separation between the user interface, the business logic, and the AI engine, which is known as the MVC pattern.

Angular (Frontend): This framework has been chosen for building a responsive, single page application (SPA). This allows for the development of specific and sophisticated user interfaces for Patients, Doctors, and Admins, ensuring that complex information is represented in a user-friendly way for non-technical users.

Node.js (Backend API): This has been chosen as the main API gateway, which handles critical backend functionalities like user authentication and doctor allocation logic. This handles the flow of information between the user and the PostgreSQL database, which stores user profiles and medical records.

FastAPI (Model Service): This is a high performance Python framework that has been chosen for building a RESTful API around the AI model. This allows for the generation of a prediction result for a patient when they submit their health data, all within a 5-second window.

Docker: All microservices have been containerized, which allows for a consistent development and production environment, removing deployment issues and ensuring that the entire system is portable.

6.3 Third party components and libraries

To ensure that high-grade output is achieved, some of the prominent libraries that are used are Scikit learn, which is used as the main driver for the Random Forest algorithm and output assessment. For data manipulation and preprocessing of the Kaggle data, Pandas and NumPy are used for missing data handling and normalization of the data. On the security front, JSON Web Tokens are used for role based user authentication so that the reports are accessible by the assigned medical staff

6.4 Algorithms

The algorithm used by the system is based on the Random Forest algorithm, which is an ensemble learning method used for performing multi class classification for the stages of lung cancer.

The reason why the Random Forest algorithm is used over other algorithms, such as simple Decision Trees or even Logistic Regression, is based on its robustness within a clinical environment. As the algorithm works by creating multiple decision trees and returning the majority "vote" of all the classes, this helps to prevent overfitting of the data by the algorithm. This is an important factor for the OncoStage AI system, as the ability to generalize well to new data is critical for the system's performance. Additionally, this algorithm is very good at recognizing complex patterns within structured clinical data, such as the association between smoking history, age, and certain symptoms, to give a confident prediction of the cancer stage.

7 Implementation Progress

7.1 Development Environment Set up

In order to establish a solid and extensible software development process for the OncoStage AI project, a modern full stack development environment has been configured. It emphasizes type safety, containerized consistency, and database management efficiency.

- Prerequisites & Runtimes

The project requires Node.js version 18 or higher to enable the asynchronous nature of the backend API as well as the Angular compilation pipeline. It also requires a relational database like PostgreSQL to manage the structured storage necessary for medical records and user profiles.

- Monorepo Configuration

The project is a monorepo, which enables the versioning of the Angular 20+ frontend as well as the Node.js Express backend. It uses a shared NPM workspace to manage the project's dependencies.

- Database ORM & Migration

Prisma ORM has been incorporated into the project to enable type safe database interactions. As part of the project's setup, automated database migration scripts (`npx prisma migrate dev`) are provided to ensure database schema consistency across developer environments.

- Environment Variables

The project's security posture is ensured by separating sensitive database credentials, JWT secrets, etc., into localized `.env` files.

7.2 Implemented Features

Implementation has been carried out according to the Agile methodology. The basic infrastructure and necessary user workflow are now fully functional.

7.2.1 Authentication & Security

- Secure Authentication

The application has a secure authentication process through the use of JWT-based tokens and the Bcrypt library for password hashing.

- Role-Based Access Control (RBAC)

The application has successfully implemented the differentiation of roles, including ADMIN, DOCTOR, PATIENT, and restricted access accordingly.

- Profile Management

Users have the functional capability to manage their profiles, including editing profiles, password management, and deleting the account.

7.2.2 Healthcare Management

- Doctor-Patient Assignment

The application has successfully implemented the logic for patient-initiated assignment. The patient can view the list of medical professionals and can assign themselves under the care of a particular doctor.

- Tailored Dashboards

Patient Dashboard

The application has provided users with the functional capability to view the doctor they are assigned to.

- Doctor Dashboard

The application has provided medical professionals with an overview of the list of patients assigned to them.

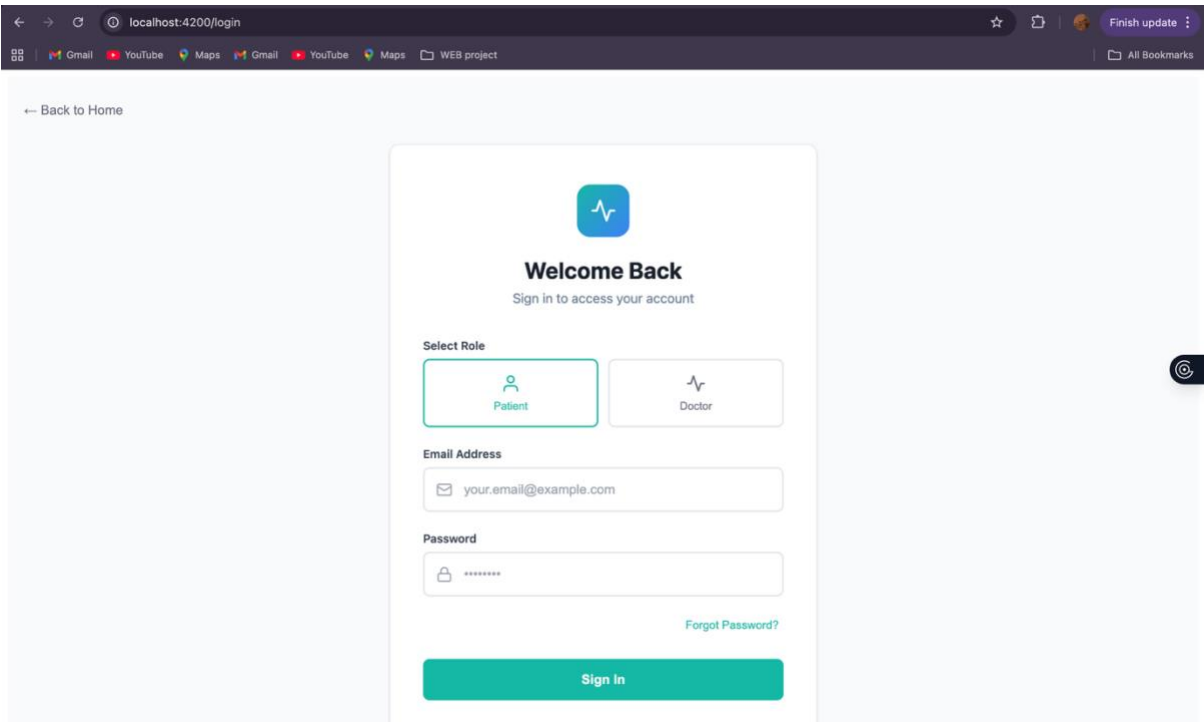
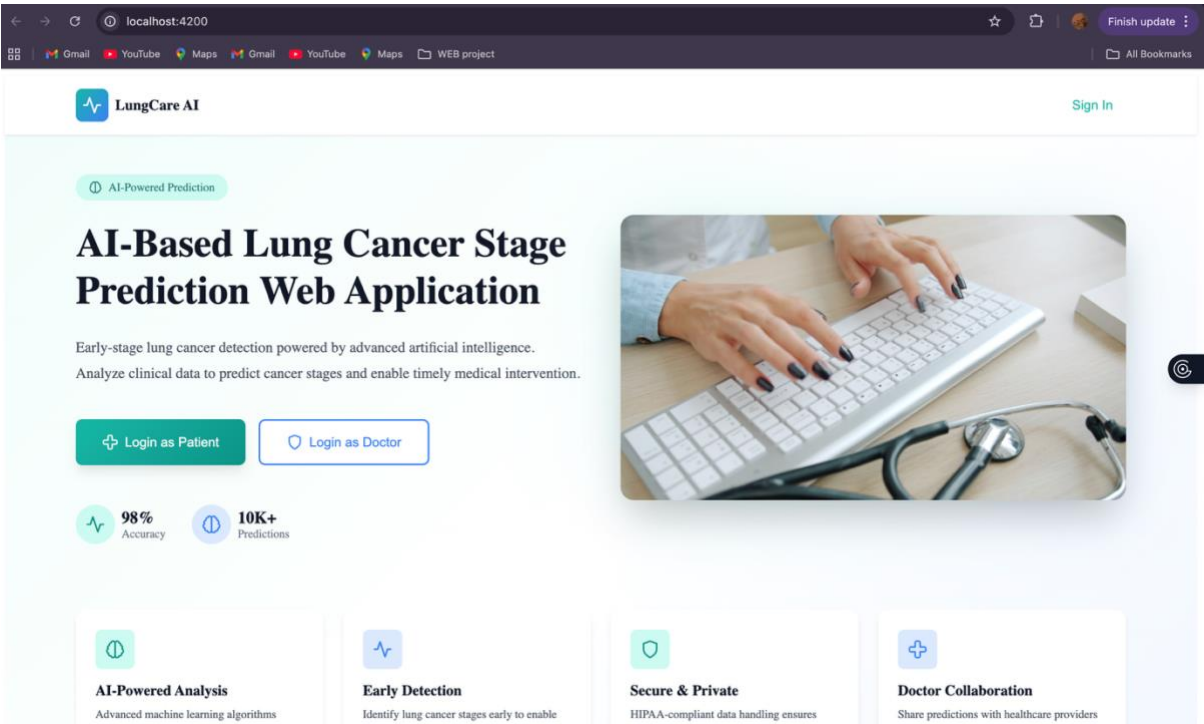
- Admin Dashboard

The application has provided the overall system overview.

7.2.3 Backend & API Integration

- Database Connectivity: The backend has successfully integrated connectivity to the PostgreSQL database through the Prisma layer, supporting CRUD operations for all primary entities.
- API Gateway: The Node.js/Express architecture is fully configured, defining routes for authentication, user management, and healthcare coordination.

7.3 Screenshots



Health Data Input Form

Please provide accurate information for AI analysis

 Demographics

Age

45

Gender

Select Gender

 Smoking History

Smoking Status

Select Status

 Symptoms

Select all symptoms you are currently experiencing:

☐ Persistent Cough☐ Shortness of Breath☐ Chest Pain☐ Weight Loss

Please fill in this field.

Hemoglobin (g/dL)

14.5

White Blood Cells ($\times 10^3/\mu\text{L}$)

7.5

Platelets ($\times 10^3/\mu\text{L}$)

250

Chest X-ray Result

Select Result

CT Scan Result (if available)

Not Performed / Select Result

 Lifestyle & Environmental Exposure

Occupational / Environmental Risk Factors

Describe any occupational hazards (e.g., asbestos, radon, chemical exposure) or environmental factors...

Ready to submit?

AI analysis will be generated based on your data

 Generate Prediction



Select Your Doctor

Choose a doctor from the list below. Each doctor can have a maximum of 10 patients per week.

JS

10 / 10 slots

Dr. John Smith

dr.smith@students.nsbm.ac.ik

Select Doctor

EJ

10 / 10 slots

Dr. Emily Johnson

dr.johnson@students.nsbm.ac.ik

Select Doctor

MW

10 / 10 slots

Dr. Michael Williams

dr.williams@students.nsbm.ac.ik

Select Doctor

SB

10 / 10 slots

Dr. Sarah Brown

dr.brown@students.nsbm.ac.ik

Select Doctor

DJ

10 / 10 slots

Dr. David Jones

dr.jones@students.nsbm.ac.ik

Select Doctor

MG

10 / 10 slots

Dr. Maria Garcia

dr.garcia@students.nsbm.ac.ik

Select Doctor



Settings

Manage your account preferences and security

- Profile
- Security
- Notifications
- Privacy

Privacy & Data

Data Protection

Your health data is encrypted and stored securely in compliance with HIPAA regulations. We never share your personal information without your explicit consent.

Data Management

Download My Data

Export all your data in a portable format

Delete Account

Permanently delete your account and all data

Legal Documents

Privacy Policy

Learn how we protect your data

Prisma Studio

Schema: public

Studio
Visualizer
Console
Tables
_prisma_migrations
doctor_patient_assignme
users

	id int4	createdAt timestamp	email text	firstName text	isActive bool	lastName text
☐	2	2026-02-11T08:48:56.728Z	dr.smith@students.nsbm.ac.lk	John	true	Smith
☐	3	2026-02-11T08:48:56.738Z	dr.johnson@students.nsbm.ac.lk	Emily	true	Johnson
☐	4	2026-02-11T08:48:56.743Z	dr.williams@students.nsbm.ac.lk	Michael	true	Williams
☐	5	2026-02-11T08:48:56.748Z	dr.brown@students.nsbm.ac.lk	Sarah	true	Brown
☐	6	2026-02-11T08:48:56.752Z	dr.jones@students.nsbm.ac.lk	David	true	Jones
☐	7	2026-02-11T08:48:56.758Z	dr.garcia@students.nsbm.ac.lk	Maria	true	Garcia
☐	8	2026-02-11T08:48:56.761Z	dr.martinez@students.nsbm.ac.lk	Carlos	true	Martinez
☐	9	2026-02-11T08:48:56.765Z	dr.rodriguez@students.nsbm.ac.lk	Ana	true	Rodriguez
☐	10	2026-02-11T08:48:56.768Z	admin@oncostage.com	Admin	true	User
☐	11	2026-02-11T08:48:56.771Z	patient@test.com	Test	true	Patient
☐	12	2026-02-11T08:50:31.430Z	arnanayakka@students.nsbm.ac.lk	Dr.Reshana	true	Nanayakkara
☐	13	2026-02-11T09:29:40.705Z	abcc@gmail.com	Test	true	Patient
☐	14	2026-02-11T09:48:44.481Z	yashiradesilva@gmail.com	yashira	true	de silva
☐	15	2026-02-15T09:43:46.861Z	ashinireshana@gmail.com	Ashini	true	Nanayakkara
☐	16	2026-03-02T03:24:56.554Z	ashini@gmail.com	ash	true	nana

colab.research.google.com/drive/1QnCO3ccw8XK1mRPtBjLD5_lrhZv_Z_m#scrollTo=kC3n6rPq8RQI

LungCancer.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text + Run all

```

Comorbidity_Autoimmune_Disease', 'Comorbidity_Other',
'Performance_Status', 'Blood_Pressure_Systolic',
'Blood_Pressure_Diastolic', 'Blood_Pressure_Pulse', 'Hemoglobin_Level',
'White_Blood_Cell_Count', 'Platelet_Count', 'Albumin_Level',
'Alkaline_Phosphatase_Level', 'Alanine_Aminotransferase_Level',
'Aspartate_Aminotransferase_Level', 'Creatinine_Level', 'LDH_Level',
'Calcium_Level', 'Phosphorus_Level', 'Glucose_Level', 'Potassium_Level',
'Sodium_Level', 'Smoking_Pack_Years', 'Symptom_Smoking',
'Yellow_Fingers', 'Anxiety', 'Peer_Pressure', 'Chronic_Disease',
'Fatigue', 'Allergy', 'Wheezing', 'Alcohol_Consuming', 'Coughing',
'Shortness_Of_Breath', 'Swallowing_Difficulty', 'Chest_Pain',
'Lung_Cancer_Symptom_Label'],
dtype='object')

X = X.drop("Patient_ID", axis=1)

for column in X.columns:
    if X[column].dtype == 'object':
        X[column] = X[column].map({"Yes": 1, "No": 0})

print(X.dtypes)
Age                                int64
Gender                             float64
Smoking_History                     float64
Tumor_Size_mm                       float64
Tumor_Location                      float64
Treatment                           float64
Survival_Months                     int64
Ethnicity                           float64
Insurance_Type                       float64
Family_History                      int64
Comorbidity_Diabetes                 int64
Comorbidity_Hypertension             int64

```

Variables Terminal

7.4 Challenges Encountered and Solutions

This section describes the technical challenges faced during the initial sprints and how they were addressed.

7.4.1 Handling Imbalanced Medical Data:

Challenge: The initial data analysis of the lung cancer dataset showed a large imbalance between the various stages of cancer growth.

Solution: The SMOTE algorithm was applied during the model training phase to ensure that all stages were equally represented.

7.4.2 Managing Doctor Patient Balancing:

Challenge: The real time constraint of not letting a patient select a doctor if the doctor has already reached his/her weekly limit of 10 patients.

Solution: A backend middleware was created using Node.js to query the PostgreSQL database for any active doctor patient assignments within a 7 day period.

7.4.3 Asynchronous AI Inference:

Challenge: The large health forms on the frontend could potentially cause the page to hang as it waited for a response from the Python model.

Solution: The AI engine was split into a separate FastAPI service that allowed for asynchronous requests.

7.5 Current System Limitations

Although the core infrastructure is up and running, the following limitations are present with the current prototype phase for OncoStage AI:

- Limited Dataset Scope

The current Random Forest model only considers structured clinical data from the Kaggle platform. It does not yet include medical image analysis, such as interpreting CT scans/X-rays, which can be part of the complete clinical staging process.

- Manual Doctor Selection

The current system requires manual selection of doctors for the patient. It does not yet include the "smart matching" feature based on the selected doctor's specialty or wait times, excluding the 10-patient wait time.

- Simulated Communication

Although the chat and video call features are implemented for the UI, the backend only supports text messaging for the time being, with the high bandwidth video conferencing feature simulated via the WebRTC placeholders for the current phase.

- Offline Training

The AI model is trained offline, and then serialization of the model is achieved by Pickle serialization. "Online learning," or the ability to update the model in real time as new data comes in from doctors, is not yet supported.

- Geographic Restrictions

The system is currently optimized for English speaking users, and there are no current plans to support multiple language localizations for a diverse range of patient demographics.

8 Discussion

8.1 Summary of the Report

In this report, the development of a clinical decision support system, OncoStage AI, to stage lung cancer based on structured data is discussed. By leveraging clinical attributes, the system can offer a low-resource diagnostic alternative to expensive image-based diagnostics. So far, the Angular frontend, Node.js backend, and PostgreSQL database are operational, offering a secure dashboard for patients and doctors to interact with the system, each according to their role.

8.2 What has Changed from the proposal

A number of changes were introduced during the implementation stage to make it more useful and robust:

- **Doctor-Patient Allocation Logic:** A specific constraint was introduced to restrict each doctor to 10 patients per week.
- **Integrated Communication:** The scope of the project was extended to include features such as chat and video calling, allowing for immediate follow ups.
- **Monorepo Architecture:** The system architecture was improved to use Prisma ORM for database management, taking it beyond the basic SQL structure initially proposed.
- **Data Sourcing:** The project was changed to use Kaggle datasets for ethical purposes and to expedite prototyping given the constraints of the university.

8.3 Future Plans

The last part of the project entails the complete integration and medical validation of the AI engine:

Model Integration: Completing the integration between the Node.js API and the FastAPI/Random Forest model for real-time inference.

System Testing: Completing unit and user acceptance testing (UAT) for the role restriction feature.

Medical Validation: Working with medical professionals to validate the accuracy of the medical reports generated by the AI model and the Random Forest model.

9 References

- American Cancer Society (2023) *Lung cancer early detection, diagnosis, and staging*. Available at: <https://www.cancer.org/> (Accessed: 10 January 2026).
- Bray, F., Ferlay, J., Soerjomataram, I., Siegel, R. L., Torre, L. A. and Jemal, A. (2021) 'Global cancer statistics 2020: GLOBOCAN estimates of incidence and mortality worldwide for 36 cancers in 185 countries', *CA: A Cancer Journal for Clinicians*, 71(3), pp. 209-249. doi: 10.3322/caac.21660.
- Breiman, L. (2001) 'Random forests', *Machine Learning*, 45(1), pp. 5-32. doi: 10.1023/A:1010933404324.
- Chen, M., Hao, Y., Cai, Y., Wang, Y. and Li, Y. (2022) 'Machine learning-based prediction models for lung cancer: A systematic review', *BMC Cancer*, 22(1), pp. 1-14. doi: 10.1186/s12885-022-09503-z.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, pp. 785-794. doi: 10.1145/2939672.2939785.
- Choudhury, A. et al. (2020) 'Lung cancer prediction using machine learning techniques', *International Journal of Engineering and Advanced Technology*, 9(3), pp. 2273-2278.
- Esteva, A. et al. (2019) 'A guide to deep learning in healthcare', *Nature Medicine*, 25, pp. 24-29. doi: 10.1038/s41591-018-0316-z.
- Kaur, H. and Sharma, A. (2020) 'Prediction of lung cancer using machine learning techniques: A review', *International Journal of Computer Applications*, 177(40), pp. 20-26. doi: 10.5120/ijca2020919933.
- Kourou, K. et al. (2015) 'Machine learning applications in cancer prognosis and prediction', *Computational and Structural Biotechnology Journal*, 13, pp. 8-17. doi: 10.1016/j.csbj.2014.11.005.
- Rajkomar, A., Dean, J. and Kohane, I. (2019) 'Machine learning in medicine', *New England Journal of Medicine*, 380(14), pp. 1347-1358. doi: 10.1056/NEJMr1814259.
- World Health Organization (2023) *Cancer: Lung cancer factsheet*. Available at: <https://www.who.int> (Accessed: 10 January 2026).
- Pedregosa, F. et al. (2011) 'Scikit-learn: Machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825-2830.
- Prisma (2026) *Prisma Documentation: Type-safe ORM for Node.js & TypeScript*. Available at: <https://www.prisma.io/docs> (Accessed: 2 March 2026).